Spring Security

Complete Learning Notes — Part 2

1. What is the http Variable in SecurityConfig

HttpSecurity is a class provided by Spring Security. When you write your SecurityFilterChain method, Spring Security automatically passes you an object of this class. That object is what you use to configure all the security rules.

Think of it like this. Spring Security hands you a settings panel and says here, use this to tell me how you want security to work. That settings panel is the HttpSecurity object. You call methods on it like csrf disable or sessionManagement or authorizeHttpRequests and each one is basically you clicking a setting on that panel.

The variable is named http just by convention. You could name it anything. But developers name it http because it represents the configuration of how your application handles incoming HTTP requests.

> *http is not something mysterious. It is just an object Spring Security gives you that represents all the configurable security behavior of your application. You use it to tell Spring Security exactly how you want things to work instead of using its defaults.*

2. Why Securing Anything Was Needed at All

When the internet started, websites were simple. Just pages of text and images. Anyone could read them. There was nothing personal, nothing sensitive, no accounts, no data. Security was not a concern because there was nothing to protect.

Then websites started becoming applications. Banks came online. Emails came online. Shopping came online. Suddenly your personal data, your money, your private messages were sitting on a server somewhere. And anyone who could send an HTTP request to that server could potentially access it.

So the need arose to answer one basic question before processing any request. Who are you and should I trust you. That is the entire reason security exists.

3. How Authentication Evolved — The Full Story

Stage 1 — Basic Auth (The Beginning)

The very first and simplest solution was to just attach your username and password to every request. Server checks them every time. If correct, process the request. If wrong, reject it. This is httpBasic.

But it had serious problems. Your username and password travel with every single request every single time. If anyone intercepts even one request they have your credentials forever. There is no expiry. There is no way to say this access is only valid for one hour. You either have

the password or you do not. Changing access means changing the password. It is like giving someone the key to your house and the only way to stop them is to change the lock.

So this worked for very simple internal systems but was clearly not good enough for real user facing applications.

Stage 2 — Sessions and Cookies

As websites grew more complex, developers thought of a better approach. Instead of sending credentials every single time, what if the user proves themselves once and the server remembers them for a while. This is how sessions were born.

User sends username and password once. Server verifies them. Server creates a session, which is basically a record stored in the server's memory that says user John is authenticated and his session ID is abc123. Server sends that session ID to the browser as a cookie. Browser stores it automatically. Every request after that just sends the cookie. Server looks up the session ID, finds John in its records, and knows who he is without asking for credentials again.

This was a huge improvement. Credentials only travel once. After that just a session ID. Much safer. This became the standard for years. Every website in the 2000s and early 2010s worked exactly like this.

But this also had problems that became more visible as applications grew. The server has to store every session in memory or database. If you have a million users logged in at the same time, the server is storing a million sessions. If the server crashes, all sessions are lost and everyone gets logged out. If you want to scale your application across multiple servers, you have a problem because server A has the session but the user's request might go to server B which has no idea who they are. Also CSRF attacks became possible because of this cookie based approach.

Stage 3 — Tokens and JWT

As applications grew even more complex, especially with mobile apps and multiple frontends, developers needed something better. Something where the server does not need to remember anything. Something that works the same whether the user is on a browser, a mobile app, or another service calling your API.

The idea was simple. Instead of the server remembering who you are, what if we give you a small document that proves who you are, and you carry that document yourself and show it whenever needed. This is JWT — JSON Web Token.

User sends username and password once to a login endpoint. Server verifies them. Instead of creating a session, the server creates a small encoded string that contains your user ID, your role, and an expiry time, and signs it with a secret key. This signed string is the JWT token. Server sends it back to you and forgets about you completely.

You store the token yourself in your React app's memory. Every request you manually attach it in the Authorization header. Server receives the request, reads the token, verifies the signature to make sure nobody tampered with it, reads your user ID and role from inside the token, and processes the request. No session lookup. No database call to check who you are. Everything the server needs is inside the token itself.

This solved all the problems of sessions. Server stores nothing. Scales perfectly across multiple servers because every server can verify the token independently. No CSRF risk because tokens are in headers not cookies and browsers never automatically send headers. Works identically for browser apps, mobile apps, and service to service communication.

4. Why Old Things Are Not Needed Anymore

formLogin is not needed because there is no session to create after login. The whole point of formLogin was to establish a session. Without sessions it has no purpose.

Sessions are not needed because the token itself carries everything the server needs. The server does not need to remember you.

CSRF protection is not needed because you are not using cookies for authentication. The entire CSRF problem existed because of automatic cookie sending. Without cookies in the authentication flow, CSRF attacks cannot work.

httpBasic is not needed in production because sending raw credentials on every request is less secure than sending a token that expires. But it stays useful for quick testing and learning.

5. The Authorization Header — What It Is and How It Evolved

In the earliest days with httpBasic, credentials traveled in the Authorization header like this.

```
Authorization: Basic dXNlcjpwYXNzd29yZA==
```

That encoded string is just your username and password combined and encoded. Server decodes it and checks.

With JWT the same header is used but now carries a token instead of credentials.

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

The word Bearer just means I am the bearer of this token, here it is. Server reads it, verifies it, and knows who you are. So the Authorization header is not a new concept. It was always there from the httpBasic days. JWT just uses the same header in a much better way. Instead of raw credentials it carries a secure signed token.

6. formLogin vs httpBasic — The Difference

Credentials used in a form in the browser is formLogin and same credentials entered in Postman in the header is httpBasic.

formLogin

This is specifically designed for the traditional web app world. It gives you a login page rendered by the server. You fill the form, submit it, the server creates a session, the browser stores a cookie, and from that point the session cookie is what proves who you are on every request. The end result of formLogin is always a session.

httpBasic

This does not care about sessions at all. It does not try to create one. It does not need one. Every single request just carries the credentials directly in the header and the server checks them right there on that request and either allows or rejects it. No session involved at any point. This is perfectly compatible with stateless.

Why mentor removed formLogin but kept httpBasic

Your mentor already set SessionCreationPolicy to STATELESS which means he told Spring Security to never create sessions under any circumstance. So even if someone successfully logs in through the form, no session gets created after that. The user would be stuck. Logged in but with nothing to prove it on the next request. formLogin becomes completely pointless because the one thing it was supposed to create, which is a session, is now disabled. It is like a door that opens into a room that does not exist.

httpBasic on the other hand works perfectly without sessions. So he kept httpBasic purely as a temporary way to test APIs through Postman while JWT is not set up yet. Once JWT arrives in Section 22, httpBasic also goes away and the Authorization header will carry a token instead of raw credentials.

7. How the Server Checks Credentials

In the current learning project

Right now your mentor is using Spring Security's default in-memory user. When the application starts, Spring Security automatically creates one user in memory with username user and a randomly generated password that prints in the console. When you send a request with httpBasic, Spring Security takes the username and password from the header, compares them against this in-memory user, and if they match it allows the request. This is only for learning purposes. The user exists only in memory and disappears the moment the server restarts.

In a real application

In a real application credentials are stored in a database. You have a users table with columns like ID, username, and password. The password is never stored as plain text. It is stored in an encrypted form using something called BCrypt. When a request comes in with credentials, Spring Security takes the username, looks it up in the database, finds the stored encrypted password, and then uses BCrypt to check if the password you sent matches the encrypted one stored. If yes it allows the request. If no it rejects it.

How Spring Security knows where to look

Spring Security does not automatically know about your database. You have to tell it by creating something called a UserDetailsService. This is a class where you tell Spring Security how to find a user. You write the logic to look up the user from your database by username and return their details. Spring Security then uses those details to verify the credentials. Your mentor covers this in lecture 307.

Complete flow in a real application

Someone sends a request with credentials. Spring Security intercepts it. It takes the username and calls your UserDetailsService to fetch that user from the database. It gets back the user object with the encrypted password. It uses BCrypt to compare the incoming password with the stored encrypted one. If they match the request is authenticated and processed. If not it returns 401 unauthorized.

8. SecurityConfig — Three Ways to Write the Same Thing

All three styles do exactly the same thing. They configure Spring Security to disable CSRF, require authentication on every request, enable httpBasic, and make sessions stateless. The only difference is how they are written.

Style 1 — Imperative (Without Lambda)

In this style for every single configuration you create a full separate Customizer object, write an @Override method inside it, put the logic there, and pass that object to http. This is the old verbose way of writing configuration in Java before lambdas existed.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

    Customizer<CsrfConfigurer<HttpSecurity>> custCsrf = new
    Customizer<CsrfConfigurer<HttpSecurity>>() {
        @Override
        public void customize(CsrfConfigurer<HttpSecurity> configurer) {
            configurer.disable();
        }
    };
    http.csrf(custCsrf);

    Customizer<AuthorizeHttpRequestsConfigurer<HttpSecurity>
    .AuthorizationManagerRequestMatcherRegistry> custHttp = new
    Customizer<AuthorizeHttpRequestsConfigurer<HttpSecurity>
    .AuthorizationManagerRequestMatcherRegistry>() {
        @Override
        public void customize(AuthorizeHttpRequestsConfigurer<HttpSecurity>
        .AuthorizationManagerRequestMatcherRegistry registry) {
            registry.anyRequest().authenticated();
        }
    };
    http.authorizeHttpRequests(custHttp);

    return http.build();
}
```

Just to disable CSRF and secure all endpoints he had to write all of this. Every configuration needs its own object, its own override, its own method. Very long and repetitive for something so simple.

Style 2 — Lambda (http on every line)

In this style the arrow function replaces that entire object creation and override pattern. Instead of creating a separate object you write the logic inline right where it is needed using the arrow. Java introduced lambdas specifically to replace this verbose pattern and make code shorter and cleaner.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

    http.csrf(customizer -> customizer.disable());
    http.authorizeHttpRequests(request ->
request.anyRequest().authenticated());
    http.httpBasic(Customizer.withDefaults());
    http.sessionManagement(session -> session
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

```
    return http.build();
}
```

Here customizer -> customizer.disable() is doing the exact same thing as that entire custCsrf object with override and method. Take the customizer object Spring gives you and call disable on it right here. One line instead of ten. http is still written on every line separately and every line ends with a semicolon because each is still a separate statement.

Style 3 — Lambda with Builder Pattern (Final Code)

Builder pattern means you configure one object step by step by chaining methods one after another. Each method does its job and returns the same http object back so you can keep calling the next method on it using a dot. That is why http is written only once at the top and removed from every other line. That is why semicolons are removed from every line except the last one. Because this is no longer multiple separate statements. It is one single continuous chain configuring one object.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {

    http.csrf(customizer -> customizer.disable())
        .authorizeHttpRequests(request ->
request.anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults())
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    return http.build();
}
```

This is the final and most modern way. This is what you will see in every real Spring Boot project at a job.

Side by Side — Just for CSRF

Imperative style to disable CSRF.

```
Customizer<CsrfConfigurer<HttpSecurity>> custCsrf = new
Customizer<CsrfConfigurer<HttpSecurity>>() {
    @Override
    public void customize(CsrfConfigurer<HttpSecurity> configurer) {
        configurer.disable();
    }
};
http.csrf(custCsrf);
```

Lambda style to disable CSRF.

```
http.csrf(customizer -> customizer.disable())
```

> *Identical result. Completely different amount of writing.*

Why Mentor Showed All Three

Because in older Spring Boot projects and older tutorials online you will see the imperative style. If your mentor only showed you lambda style and you opened an old codebase at a job and saw all those Customizer objects and overrides you would be completely confused. Now you know

all three styles do the same thing and you can read any of them comfortably wherever you see them.

Practical Flow of Spring Security

(We have seen till CSRF — now let us see without CSRF and Sessions)

9. The Doubt That Started This

When setting up the stateless REST API config, the question was whether writing server.servlet.session.cookie.same-site=strict in application.properties is a necessary step to disable Spring Security defaults.

The answer is no. That line has nothing to do with disabling Spring Security defaults. The only thing needed to disable defaults is creating your own SecurityConfig class with SecurityFilterChain bean. That line is purely a cookie protection setting and nothing else.

10. What SameSite Strict Actually Means

SameSite is an attribute you can set on a cookie. It tells the browser one rule. Only send this cookie if the request is originating from the same site that owns the cookie. If the request is coming from a different site, do not attach the cookie at all.

11. Same Site and Cross Site — A Practical Example

You made a Spring Boot API running on localhost:8080. You open localhost:8080/someEndpoint directly in the browser address bar. The request originates from the same place it is going to. This is same site. Cookie gets attached normally.

Now in a real application your React frontend runs on localhost:3000 and your backend runs on localhost:8080. React sends a request to the backend. The browser sees the request is originating from localhost:3000 and going to localhost:8080. Different ports means different origin. This is cross site.

If SameSite strict is set, the browser refuses to attach the cookie even though it has the cookie stored. This means SameSite strict would break your own legitimate React frontend too, not just evil.com. Your own app's requests arrive at the backend with no session cookie. Backend sees no session and rejects every request with 401.

12. Why SameSite Was Introduced So Late

Cookies were invented in 1994. Browsers auto sent them everywhere for decades. CSRF attacks became a serious problem in the early 2000s. Since there was no browser level protection at that time, developers invented the CSRF token as the solution.

SameSite was only introduced in 2016 by Google. By that time CSRF token protection was already deeply embedded in every framework including Spring Security. So both solve the same problem. CSRF token is the old battle tested solution. SameSite is the modern cleaner browser level solution. Both exist because the web evolved slowly.

13. What SameSite Lax Is

Lax is a relaxed version of Strict. Strict means never send the cookie if the request comes from any other origin. Completely blocks everything cross origin.

Lax means send the cookie for normal navigation like clicking a link, but do not send it for form submissions or API calls coming from another origin. It is a middle ground. Blocks most CSRF scenarios without breaking everything.

Browsers moved to Lax as the default behavior around 2020 specifically to reduce CSRF risk without breaking legitimate cross origin navigation. So even if you do not set anything, modern browsers already apply Lax by default.

14. What CORS Is

CORS stands for Cross Origin Resource Sharing.

Same Origin Policy blocks JavaScript on one origin from reading responses from another origin. CORS is the mechanism that lets a server say I am okay with requests from this specific origin, allow it.

When your React app on localhost:3000 calls your backend on localhost:8080, the browser first checks whether the backend allows requests from localhost:3000. If CORS is not configured on the backend, the browser blocks the response even if the request actually reached the server successfully and the server processed it.

You configure CORS inside SecurityConfig in Spring Boot by specifying which origins are allowed to call your API.

15. Default CORS Behavior in Spring Boot

By default Spring Boot allows nothing from a different origin. Any request coming from React on a different port will be blocked by the browser when it tries to read the response. The backend does not explicitly allow any cross origin requests out of the box.

This means the very first time you connect your React frontend to your Spring Boot backend you will get a CORS error in the browser console. This is one of the most common errors every developer hits when building their first full stack application. It is not a bug. It is the browser enforcing Same Origin Policy because your backend has not configured which origins are allowed yet.

> *Default Spring Boot behavior is most restrictive. Same origin only. Everything else is blocked unless you explicitly configure it.*

16. How SameSite and CORS Are Different

SameSite is about cookies. It controls whether the browser attaches a cookie when making a cross origin request. It acts when the request is being sent.

CORS is about responses. It controls whether the browser allows JavaScript to read a response that came from a different origin. It acts when the response comes back.

They solve different problems at different points of the same request journey. SameSite is cookie related. CORS is completely independent of cookies.

	SameSite	CORS
What it protects	Cookies	Responses
When it acts	Request being sent	Response coming back
Related to cookies	Yes	No
Needed with JWT	No	Yes always

17. What JWT Removes and What It Does Not

JWT removes CSRF problems and SameSite complications completely because both of those are cookie related. No cookies means no CSRF risk and no SameSite needed.

But CORS is completely independent of authentication and cookies. It will come up regardless of whether you use sessions or JWT. The moment your React frontend on one origin tries to call your Spring Boot backend on a different origin, the browser will block the response unless CORS is configured. JWT does not change this at all.

> *CORS configuration is something every real project needs. JWT removes CSRF and SameSite complications but CORS remains and must always be configured.*

18. When We Say Browser — What Does That Mean

When we say browser we mean whatever client is running your frontend code. If your React app is running in Chrome on a laptop, Chrome is the browser enforcing Same Origin Policy and CORS rules. If your React app is running inside a mobile browser like Chrome on Android or Safari on iPhone, the mobile browser enforces the same rules.

But if you build a native mobile app like a proper Android app built with Kotlin or an iOS app built with Swift, those are not browsers. They are native applications. Same Origin Policy and CORS are browser rules. They do not apply to native apps at all. A native Android app can call any API from any origin freely. No CORS error. No SameSite restriction.

> *CORS is a browser rule. It does not matter whether that browser is on a laptop or a mobile phone. If a browser is involved CORS applies. If it is a native app CORS does not apply.*

19. Real World Examples

Apps that have both a browser version and a native mobile app

Gmail is a perfect example of both worlds. When you open Gmail on your laptop you use the browser version. Web frontend running in Chrome. CORS rules apply here. When you open Gmail on your phone you use the native Gmail app. This is a native Android or iOS application. It calls Google's backend API directly. No browser involved. CORS rules do not apply. Same Gmail backend API, same your emails, but two completely different clients calling it. One is a browser where CORS matters, one is a native app where it does not.

Other apps that work the same way are Swiggy, Zomato, Amazon, Flipkart, Instagram, and YouTube. All of them have a browser version where CORS applies and a native mobile app where it does not. Same backend API, different rules depending on what is calling it.

Apps that are mobile only with no browser version

CRED is a good example. It is a credit card bill payment and rewards app available only as a native Android and iOS app. There is no full browser version where you can use the product on your laptop. Their backend API is called only by native apps. CORS is completely irrelevant for them.

PhonePe is primarily a native mobile app. You cannot open PhonePe on a laptop browser and do UPI transactions the same way. Their backend is called by native apps mainly. CORS does not apply.

Meesho in its early days was heavily mobile focused with the core reseller product only on the native app.

The complete picture

The same backend API can be called by a browser on a laptop where CORS applies, a browser on a mobile phone where CORS still applies because it is still a browser, and a native mobile app on Android or iOS where CORS does not apply at all because there is no browser involved.

20. The Practical Flow — Where Everything Connects

We started with normal username and password to protect data. Then session concept was introduced so credentials do not travel every time. Then with sessions, CSRF attacks were discovered. Three solutions came up to fight CSRF. Either use a CSRF token, or apply SameSite rule on cookies, or make the app stateless so there are no sessions and no cookies and therefore no CSRF problem at all.

To go stateless two things are necessary. Disable CSRF because no cookies means no need for it. Set session policy to stateless so no sessions are created.

Once stateless, formLogin has no purpose because formLogin's entire job is to create a session after login. No sessions means formLogin is pointless. So it gets removed.

httpBasic stays for now as a temporary testing mechanism because it works without sessions. It just sends username and password directly in the header on every request from Postman. No session involved. No browser form involved. Just raw credentials in the header for testing purposes until JWT replaces it.

Right now without JWT the flow is incomplete. httpBasic is just a temporary bridge. Every request still carries raw username and password which is not ideal. JWT will replace this properly. After login you get a token, store it, and send it in the Authorization header on every request. No credentials traveling again. No sessions. No CSRF. Clean and complete. That is exactly what Section 22 of your course covers.

Practical Doubts and Results

What each code configuration gives and why

21. Code 1 — Only CSRF Disabled, No Auth Rules

```
http.csrf(customizer -> customizer.disable());
return http.build();
```

What happens

GET request works fine. No authentication needed because CSRF does not apply to GET requests. POST request gives 403 Forbidden because CSRF is still enabled by default for state changing methods and no token is sent.

Why

You only disabled CSRF. Spring Security's default behavior is still active. GET requests are safe methods so they pass through. POST, PUT, DELETE are state changing so the CSRF filter blocks them with 403.

22. Code 2 — authorizeHttpRequests Added But No Auth Mechanism

```
http.csrf(customizer -> customizer.disable());
http.authorizeHttpRequests(request -> request.anyRequest().authenticated());
return http.build();
```

Doubt that came up

Username and password were provided in Postman but still getting 403. Why is it not working even after sending credentials?

What happens

403 Forbidden on both browser and Postman even when credentials are provided.

Why

You told Spring Security every request needs authentication but you did not tell it how to authenticate. You provided username and password but Spring Security does not know where to look for them. Is it in a form, in a Basic Auth header, in a JWT token? No authentication mechanism is configured so Spring Security simply blocks everything regardless of what credentials you send.

> *authorizeHttpRequests defines the rules. But without an auth mechanism like httpBasic or formLogin, Spring Security has no way to verify the credentials even if they are present in the request.*

23. Code 3 — formLogin Added

```
http.csrf(customizer -> customizer.disable());
http.authorizeHttpRequests(request -> request.anyRequest().authenticated());
http.formLogin(Customizer.withDefaults());
return http.build();
```

Doubt that came up

This worked on browser but Postman was returning HTML instead of JSON data. Why is the response HTML?

What happens on browser

Spring Security redirects you to an HTML login page. You fill credentials, session is created, you can access endpoints normally.

What happens on Postman

Response comes back as HTML of the login page instead of actual JSON data. Status shows 200 OK but the body is the login page HTML with Please sign in heading and a form with username and password fields.

Why

formLogin is designed for browser based form submissions. When Postman hits an endpoint without authentication, Spring Security redirects to the login page. Postman follows that redirect and shows you the HTML of the login page. The response is 200 OK because Postman successfully loaded the login page. But that is not your actual data. formLogin is not suitable for REST APIs because it returns HTML redirects instead of JSON responses.

> *A REST API should return 401 unauthorized in JSON format when credentials are missing, not redirect to an HTML login page. formLogin does the opposite of what a REST API needs.*

24. Code 4 — formLogin and httpBasic Both Added with STATELESS

```
http.csrf(customizer -> customizer.disable());
http.authorizeHttpRequests(request -> request.anyRequest().authenticated());
http.formLogin(Customizer.withDefaults());
http.httpBasic(Customizer.withDefaults());
http.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
return http.build();
```

What happens on Postman

Works perfectly. Returns actual JSON data every time.

What happens on browser

Shows login form. You fill credentials. Spring Security verifies them correctly. But then it redirects to login form again. You fill credentials again. Same thing. Infinite loop. You can never get through.

Doubt that came up

Postman is working but browser keeps showing login form even after filling correct credentials. Why?

Why browser loops

formLogin's entire purpose is to create a session after successful login so the browser can use a cookie for every subsequent request. But you set SessionCreationPolicy to STATELESS which means Spring Security will never create a session under any circumstance. So after you log in successfully, Spring Security has nowhere to store that you are logged in. The next request comes in, no session found, redirects to login form again. Infinite loop.

> *formLogin needs sessions to work. STATELESS removes sessions. These two directly contradict each other and can never be used together.*

25. Code 5 — formLogin Removed, Only httpBasic with STATELESS (Final Working Code)

```
http.csrf(customizer -> customizer.disable());
http.authorizeHttpRequests(request -> request.anyRequest().authenticated());
http.httpBasic(Customizer.withDefaults());
http.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS));
return http.build();
```

What happens on Postman

Works perfectly. Returns actual JSON data every time with no issues.

What happens on browser

Browser shows a built in username password popup. Not a Spring Security page. The browser's own native popup. You fill credentials once. Request goes through and returns actual data. When you hit different endpoints, no popup appears again and everything works seamlessly.

Doubt — Why does browser not ask for credentials again after filling once

This is not Spring Security remembering you. This is the browser remembering your credentials. When httpBasic is enabled, Spring Security returns a WWW-Authenticate header with 401 when credentials are missing. Browser sees this header and shows its built in Basic Auth popup. After you fill credentials once, the browser caches them temporarily for that domain and silently sends them in the Authorization header with every subsequent request automatically. Spring Security on the backend is still authenticating every single request independently from scratch. It knows nothing about previous requests. The browser is just making it feel seamless by sending credentials silently every time.

> *Spring Security stores nothing. The browser stores credentials temporarily for convenience. These are two completely different things.*

Doubt — Why is the browser showing a popup and not an HTML form

Because you removed formLogin and kept only httpBasic. formLogin generates an HTML login page. httpBasic does not generate any page. Instead it sends a WWW-Authenticate response

header which is a signal that tells the browser to show its own built in credential popup. That popup is entirely the browser's own UI, not anything Spring Security generated.

Doubt — Session IDs are different for different endpoints, why

Because sessions are STATELESS. No session is ever created or stored on the server. Every request is a completely fresh authentication. If a session ID appears anywhere it is meaningless because it is never actually used for authentication. Spring Security verifies credentials fresh on every single request and immediately forgets after processing. This is exactly what stateless means.

Doubt — If sessions are not created where is the browser storing them

The browser is not storing a session at all. The browser is storing the credentials themselves, meaning your username and password. This is built in browser behavior specifically for Basic Auth. When you fill the popup, the browser saves those credentials in its own internal memory for that domain and port. Not as a cookie, not as a session. Just credentials cached for that origin. Every request to the same origin the browser takes those saved credentials, encodes them, and puts them in the Authorization header automatically. You can verify this yourself. Open a new incognito window and hit the same endpoint. The browser will ask for credentials again because incognito does not share the credential cache with the normal window.

26. Quick Reference — What Code Gives What Result

Configuration	Result
Only CSRF disabled	GET works, POST gives 403
anyRequest authenticated, no auth mechanism	403 everywhere even with credentials provided
formLogin added	Browser works, Postman returns HTML login page instead of JSON
formLogin + httpBasic + STATELESS	Postman works, browser loops forever on login form
httpBasic + STATELESS only	Both Postman and browser work correctly

> *formLogin needs sessions. httpBasic does not need sessions. STATELESS removes sessions. So formLogin plus STATELESS always breaks. httpBasic plus STATELESS always works. This is the foundation of why REST APIs use httpBasic temporarily and JWT permanently instead of formLogin.*

How to Convert Imperative to Lambda

The general rule with examples

27. The General Rule

In imperative style you are creating an anonymous class that implements an interface and overriding one method inside it. Lambda is just a shortcut to write that same thing without all the boilerplate.

The rule is simple. Whatever was inside the customize method body becomes the right side of the arrow. Whatever parameter the customize method received becomes the left side of the arrow.

Imperative always looks like this

```
http.someMethod(someObject);

// Where someObject is a full anonymous class like this:
new SomeInterface() {
    @Override
    public void someMethodName(ParameterType parameter) {
        parameter.doSomething();
    }
}
```

Lambda collapses it to this

```
http.someMethod(parameter -> parameter.doSomething());
```

Everything in the middle disappears. Only the parameter and what you do with it survives. Connected by an arrow.

28. Four Steps to Convert Any Imperative to Lambda

Step 1. Keep the http.someMethod() part exactly as it is.

Step 2. Find the parameter name inside the customize method signature.

Step 3. Find what is written inside the method body.

Step 4. Write parameter arrow body and pass it directly inside the http.someMethod().

> *Find the parameter. Find what it does. Connect them with an arrow. Pass it to the same http method. That is the entire rule.*

29. Example 1 — CSRF

Imperative

```
Customizer<CsrfConfigurer<HttpSecurity>> custCsrf =
    new Customizer<CsrfConfigurer<HttpSecurity>>() {
        @Override
        public void customize(CsrfConfigurer<HttpSecurity>
httpSecurityCsrfConfigurer) {
            httpSecurityCsrfConfigurer.disable();
        }
    };
http.csrf(custCsrf);
```

Applying the rule

Step 1. http method is http.csrf() so that stays.

Step 2. Parameter name is httpSecurityCsrfConfigurer.

Step 3. Body is httpSecurityCsrfConfigurer.disable().

Step 4. Connect them with arrow and pass inside http.csrf().

Lambda

```
http.csrf(httpSecurityCsrfConfigurer -> httpSecurityCsrfConfigurer.disable());
```

> *Your mentor renamed the parameter to customizer to make it shorter. The name does not matter. You can name it anything. http.csrf(customizer -> customizer.disable()) and http.csrf(x -> x.disable()) both do exactly the same thing.*

30. Example 2 — authorizeHttpRequests

Imperative

```
Customizer<AuthorizeHttpRequestsConfigurer<HttpSecurity>
.AuthorizationManagerRequestMatcherRegistry> custHttp = new
Customizer<AuthorizeHttpRequestsConfigurer<HttpSecurity>
.AuthorizationManagerRequestMatcherRegistry>() {
    @Override
    public void customize(AuthorizeHttpRequestsConfigurer<HttpSecurity>
    .AuthorizationManagerRequestMatcherRegistry registry) {
        registry.anyRequest().authenticated();
    }
};
http.authorizeHttpRequests(custHttp);
```

Applying the rule

Step 1. http method is http.authorizeHttpRequests() so that stays.

Step 2. Parameter name is registry.

Step 3. Body is registry.anyRequest().authenticated().

Step 4. Connect them with arrow and pass inside http.authorizeHttpRequests().

Lambda

```
http.authorizeHttpRequests(registry -> registry.anyRequest().authenticated());
```

31. Example 3 — sessionManagement

Imperative

```
Customizer<SessionManagementConfigurer<HttpSecurity>> custSession =
    new Customizer<SessionManagementConfigurer<HttpSecurity>>() {
        @Override
        public void customize(SessionManagementConfigurer<HttpSecurity>
session) {
            session.sessionCreationPolicy(SessionCreationPolicy.STATELESS);
```

```
        }
    };
http.sessionManagement(custSession);
```

Applying the rule

Step 1. http method is http.sessionManagement() so that stays.

Step 2. Parameter name is session.

Step 3. Body is session.sessionCreationPolicy(SessionCreationPolicy.STATELESS).

Step 4. Connect them with arrow and pass inside http.sessionManagement().

Lambda

```
http.sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
```

> *The parameter name you choose on the left side of the arrow has zero effect on what the code does. It is just a label you pick yourself. Short and readable is always better.*

> ***Next up — Working with Multiple Users, UserDetailsService, BCrypt, and Database Integration.***